



Exercice 1

Il n'existe pas de réponse définitive à la question de savoir si un algorithme récursif est préférable à un algorithme itératif ou le contraire. Il a été prouvé que ces deux paradigmes de programmation sont équivalents ; autrement dit, tout algorithme itératif possède une version récursive, et réciproquement. Un algorithme récursif est aussi performant qu'un algorithme itératif pour peu que le programmeur ait évité les quelques écueils qui seront étudiés plus loin et que le compilateur ou l'interprète de commande gère convenablement la récursivité. Le choix du langage peut aussi avoir son importance : un langage fonctionnel tel Caml est conçu pour exploiter la récursivité et le programmeur est naturellement amené à choisir la version récursive de l'algorithme qu'il souhaite écrire. à l'inverse, Python, même s'il l'autorise, ne favorise pas l'écriture récursive¹ (limitation basse par défaut du nombre d'appels récursifs, pas d'optimisation pour la récursivité terminale).

Enfin, le choix d'écrire une fonction récursive ou itérative peut dépendre du problème à résoudre : certains problèmes se résolvent particulièrement simplement sous forme récursive, et le plus emblématique de tous est sans conteste le problème des tours de Hanoï inventé par le mathématicien français Edouard Lucas. Ce jeu mathématique est constitué de trois tiges sur lesquelles sont enfilés n disques de diamètres différents. Au début du jeu, ces disques sont tous positionnés sur la première tige (du plus grand au plus petit) et l'objectif est de déplacer tous ces disques sur la troisième tige, en respectant les règles suivantes :

- un seul disque peut être déplacé à la fois ;
- on ne peut jamais poser un disque sur un disque de diamètre inférieur.

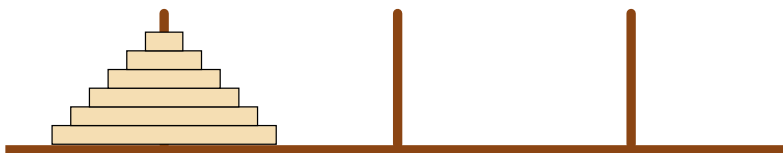


Figure 1 — Le puzzle dans sa configuration initiale



Cliquer [ici](#)

Maintenant, raisonnons par récurrence : pour pouvoir déplacer le dernier disque, il est nécessaire de déplacer les $n - 1$ disques qui le couvrent sur la tige centrale. Une fois ces déplacements effectués, nous pouvons déplacer le dernier disque sur la troisième tige. Il reste alors à déplacer les $n - 1$ autres disques vers la troisième tige.

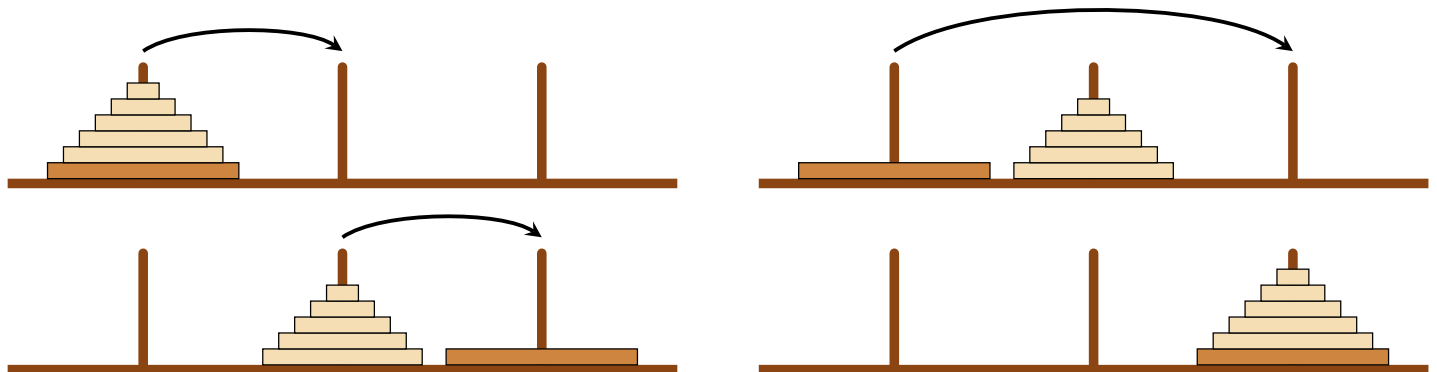


Figure 2 — Technique de résolution

Pour pouvoir déplacer n disques de la tige 1 vers la tige 3 il suffit de savoir déplacer $n - 1$ disques de la tige 1 vers la tige 2 puis de la tige 2 vers la tige 3. Autrement dit, il suffit de généraliser le problème de manière à décrire le déplacement de n disques de la tige i à la tige k en utilisant la tige j comme pivot.

```
1 Données :  
2 n: entier indiquant le nombre de disques à déplacer.  
3 dep, inter, arr : chaînes de caractères pour indiquer les tiges de départ, intermédiaire et d'  
   arrivée
```

1. On peut citer à ce sujet Guido van Rossum, le créateur du langage Python : I don't believe in recursion as the basis of all programming. This is a fundamental belief of certain computer scientists, especially those who (...) like to teach programming by starting with a "cons" cell and recursion. But to me, seeing recursion as the basis of everything else is just a nice theoretical approach to fundamental mathematics (...), not a day-to-day tool.

```

4 fonction Hanoi(n,dep,inter,arr):
5     Si n = 0
6         renvoyer Rien
7     Sinon
8         Hanoi(n - 1,dep,arr,inter)
9         afficher "déplacement d'un disque de" dep "vers" arr
10        Hanoi(n - 1,inter,dep,arr)

```

Exercice 2

Le problème des n reines consiste à placer n reines sur un échiquier de taille $n \times n$ de sorte que deux reines quelconques ne puissent jamais s'attaquer (pour ceux d'entre vous qui ne sont pas familiers avec le jeu d'échecs, ceci signifie que deux reines quelconques ne partagent jamais la même ligne, la même colonne ou la même diagonale). De telles positions seront par la suite qualifiées de **valides** (illustration figure 3).

Il est évident que dans chaque colonne doit se trouver exactement une reine ; ainsi il est possible de représenter ce problème par un tableau de n cases $q = [q_0, \dots, q_{n-1}]$ dans lequel q_j désigne la ligne où est placée la reine de la colonne j .

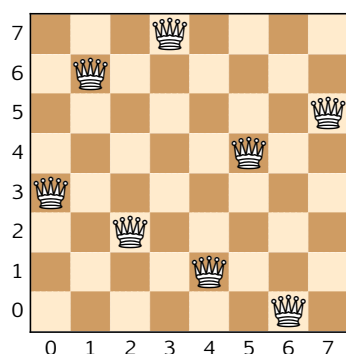


Figure 3 – Une solution, représentée par le tableau
[3,6,2,7,1,4,0,5]

On appellera **solution partielle** de rang j un tableau q de longueur n dont les j premières cases sont remplies par des positions valides pour les reines, les $n - j$ autres cases restant à remplir.

Rédiger une fonction récursive **reine**(q, j) qui prend pour arguments un entier j et une solution partielle q et qui réalise les opérations suivantes :

- si $j = n$ cette fonction se contente d'afficher le tableau q (le problème est résolu) ;
- si $j < n$, cette fonction recherche parmi les n valeurs possibles pour q_j celles qui correspondent à des positions valides et pour chacune d'elles poursuit la recherche au rang $j + 1$.

Indication

La reine de la colonne k peut attaquer la reine de la colonne j dans l'un des trois cas suivants : $q_k = q_j$ (les deux reines sont sur la même rangée), $q_j - q_k = j - k$ ou $q_j - q_k = k - j$ (les deux reines sont sur la même diagonale).



Cliquer [ici](#)

Exercice 3

Le jeu du baguenaudier est un puzzle constitué d'anneaux enchevêtrés dans une navette, le but étant de libérer celle-ci du système des anneaux. Il peut être modélisé de la manière suivante : on dispose d'une réglette contenant n cases numérotées de 1 à n et sur chacune desquelles est disposé un pion. Le but du jeu est d'ôter ces n pions en respectant les règles suivantes :

- il n'y a jamais plus d'un pion par case ;
- on peut à tout moment poser (s'il n'y en a pas) ou enlever (s'il y en a un) un pion sur la case 1 ;
- on peut poser ou enlever un pion sur la case $j \in \llbracket 2; n \rrbracket$ s'il y a un pion sur la case $j - 1$ et aucun sur les cases précédentes.



Figure 4 – Dans cette configuration, on peut poser un pion dans la case 1 ou ôter celui de la case 4

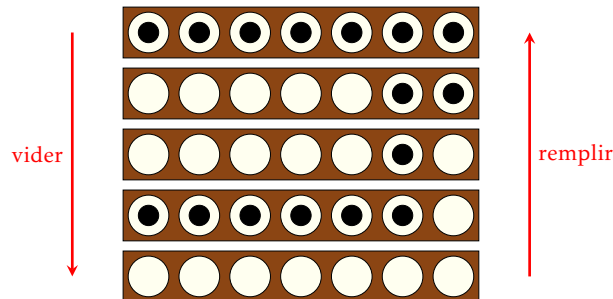


Cliquer [ici](#)

Indication

Si $n \geq 2$, on vide les n premières cases initialement pleines de la manière suivante :

- vider les $n - 2$ premières cases ;
- enlever le pion de la case n ;
- remplir les $n - 2$ premières cases ;
- vider les $n - 1$ premières cases.



De même, on remplit les n premières cases initialement vides de la manière suivante :

- remplir les $n - 1$ premières cases ;
- vider les $n - 2$ premières cases ;
- poser un pion dans la case n ;
- remplir les $n - 2$ premières cases.

- 1) Écrire une fonction `peut_jouer(etat, j)` qui renvoie `True` si l'on a le droit de jouer (poser ou retirer) sur la case `j`, `False` sinon.
- 2) Écrire deux fonctions `poser(etat, j)` et `enlever(etat, j)` qui modifient `etat` en place, en levant une erreur (`assert`) si le coup est illégal, et qui affichent le coup joué.
- 3) En s'inspirant de la structure des **Tours de Hanoi** vue précédemment, écrire deux fonctions récursives croisées :
 - `vider(etat, n)` qui vide entièrement les cases 1 à `n`, en supposant qu'elles sont toutes initialement remplies ;
 - `remplir(etat, n)` qui remplit entièrement les cases 1 à `n`, en supposant qu'elles sont toutes initialement vides.